REINFORCEMENT LEARNING FOR BLOCK BLAST

From Deep Q-Networks to Deep Value Networks

March 2026

Nathan Duboisset

CONTENTS

1 Introduction **4**

2 Environment **4**

2.1 Game Rules 4

2.2 Single-Piece Variant (1P) 5

2.3 Three-Piece Variant (3P) 5

2.4 Reward Shaping 5

2.4.1 Initial Quadratic Reward (Abandoned) 6

2.4.2 Final Reward Function (1P) 6

2.4.3 Final Reward Function (3P) 6

3 DQN-Based Approaches **6**

3.1 Architecture 6

3.2 Double DQN (DDQN) 7

3.3 Rainbow DQN 7

3.4 Limitations 7

4 Deep Value Network (DVN) **7**

4.1 Afterstate Formulation 8

4.2 Architecture: Multi-Kernel CNN 8

4.3 Training 9

4.3.1 Principle 9

4.3.2 Dynamics 9

4.4 Results 1P 10

4.5 Results 3P 12

4.5.1 Motivation and Combinatorial Explosion 12

4.5.2 Benchmark Methodology 12

4.5.3 Results and Superiority of the DVN 13

4.5.4 Computational Bottlenecks and Limitations 15

5 Proximal Policy Optimization (PPO) **16**

5.1 Principle 16

5.2 Arborescent search upgrade 17

6 Discussion and Future Work **19**

7 Conclusion **19**

A Model Architectures **21**

A.1 BlockBlastCNNNet1P (DDQN / Rainbow) 21

A.2 BlockBlastValueNet1PmultikernelFlattenned (DVN) 21

Abstract

We investigate reinforcement learning approaches to the puzzle game *Block Blast*, played on an 8×8 grid. The agent must place Tetromino-like pieces to clear full rows and columns. We study two environment variants: a single-piece variant (1P) where one piece is given at a time, and a three-piece variant (3P) where three pieces are presented simultaneously and must all be placed in a freely chosen order. We experiment with classical Deep Q-Network approaches (DDQN, Rainbow DQN) and introduce a *Deep Value Network* (DVN) that operates on *afterstates*—the board states resulting from each possible placement—instead of learning a standard Q-function. The DVN achieves significantly better performance than naive baselines with only 42 461 parameters, compared to the ∼2.67M parameters needed by DQN-based models.

1 INTRODUCTION

Block Blast is a single-player puzzle game played on an 8×8 grid. At each turn, the player receives one or more Tetromino-like pieces and must place them on the grid. When a full row or column is formed, it is cleared and points are scored. The game ends when no more pieces can be placed.

This game presents several classic challenges for reinforcement learning:

- **Long-horizon planning**: clearing lines requires setting up the board over many turns.
- **Large action space**: up to 64 positions per piece in the 1P variant, 192 in the 3P variant.
- **Sparse rewards**: most individual placements yield no line clears.
- **Combinatorial structure**: in the 3P variant, the order of the three pieces matters.

We organised our experiments in three phases. First, we designed the environment and iterated on the reward function. Second, we trained standard DQN-based agents (DDQN and Rainbow DQN). Third, we developed a Deep Value Network (DVN) operating on afterstates.

2 ENVIRONMENT

2.1 Game Rules

The game is played on an 8×8 binary grid (0 = empty, 1 = filled). Seven piece shapes are used, each randomly rotated by a multiple of 90° at the start of each episode:

Shape	Description
Dot	1×1 single cell
Square	2×2 block
Big Square	3×3 block
Line-3	1×3 horizontal strip
Line-4	1×4 horizontal strip
L-small	2×2 L-shape
L-large	3×3 L-shape

After each placement, every row and column that is entirely filled is cleared simultaneously. The agent must avoid reaching a state where no valid placement exists, as this ends the episode.

2.2 Single-Piece Variant (1P)

In the 1P variant, one piece is drawn uniformly at random at the start of each step. The observation space is a dictionary containing:

- `board`: the 8×8 integer grid,
- `piece`: the piece shape encoded in a 5×5 padded grid,
- `valid_placements`: an 8×8 binary mask of valid top-left positions,
- `placements_result`: a precomputed $(8, 8, 8, 8)$ tensor giving the resulting board for every candidate placement, alongside an $(8, 8)$ reward map.

The action space is Discrete(64), where action $a = 8r + c$ places the piece with its top-left corner at row r, column c.

2.3 Three-Piece Variant (3P)

In the 3P variant, three pieces are drawn at the start of each *round*. The agent must place all three in a freely chosen order before a new round begins. The action space becomes Discrete(192):

$$a = \text{piece_idx} \times 64 + r \times 8 + c$$

A binary vector `pieces_used` tracks which pieces have been played. A `combo` counter accumulates cleared lines and is reset when a full round completes without any clearing.

The environment also exposes `get_t_plus_3_candidates(`γ`)`, which enumerates all 3! permutations and their valid positions, returning the terminal board, cumulative discounted reward, and action sequence for each candidate. This is used by the DVN 3P planner (Section 4).

2.4 Reward Shaping

2.4.1 • Initial Quadratic Reward (Abandoned)

In early experiments, we used a quadratic bonus for clearing multiple lines at once:

$$r = b \cdot n_{\text{cleared}}^2, \qquad b = 10$$

The intuition was to strongly incentivise combos. However, clearing multiple lines simultaneously is rare early in training, making the reward signal extremely sparse. The agent received near-zero reward for the vast majority of steps, causing training to stall.

2.4.2 • Final Reward Function (1P)

The final 1P reward is computed for each hypothetical afterstate:

$$r = \underbrace{r_{\text{survive}}}_{5.0} + \underbrace{b \cdot n_{\text{cleared}}}_{10\times\#\text{lines}} + \underbrace{b \cdot \rho}_{\text{emptiness}} - \underbrace{p_{\text{no3x3}}}_{20 \text{ if no free } 3\times3} \tag{1}$$

where $\rho \in [0, 1]$ is the fraction of empty cells (encourages keeping the board clean), and $p_{\text{no3x3}} = 20$ if no 3×3 window of empty cells exists after placement. Invalid moves incur a penalty of -100 and terminate the episode.

2.4.3 • Final Reward Function (3P)

The 3P variant uses a combo-based reward:

$$r = \begin{cases} 0.1 & \text{if no lines cleared} \\ b \cdot (n_{\text{cleared}}^2 + \text{combo}) & \text{if } n_{\text{cleared}} > 0 \end{cases}$$

The `combo` counter accumulates across placements, encouraging sustained clearing streaks.

3
DQN-BASED APPROACHES

3.1 Architecture

Both DDQN and Rainbow share the same convolutional backbone, `BlockBlastCNNNet1P`:

1. **Board branch**: two channels (board + valid mask), 8×8
   $\to$ Conv2d($2 \to 32$, $k = 3$) $\to$ Conv2d($32 \to 64$, $k = 3$) $\to$ Flatten $\to$ 4096 features.
2. **Piece branch**: one channel (5×5) $\to$ Conv2d($1 \to 32$, $k = 3$) $\to$ Flatten $\to$ 800 features.
3. **FC head**: $4896 \to 512 \to 256 \to 64$ Q-values.

Total parameters: **2 674 464**.

3.2 Double DQN (DDQN)

Double DQN decouples action selection from action evaluation in the target:

$$y_t = r_t + \gamma \cdot Q_{\theta^-}\Big(s_{t+1},\ \arg\max_a Q_\theta(s_{t+1}, a)\Big)$$

where θ is the policy network and θ^- the target network (updated every 10 episodes). Invalid actions are masked with -10^9 before the argmax.

Training: ε decayed $1.0 \to 0.05$ ($\times 0.9995$/episode), replay buffer 10 000, batch size 64, $\gamma = 0.99$, Adam lr $= 10^{-4}$.

3.3 Rainbow DQN

Rainbow combines five extensions:

- **Distributional RL (C51)**: outputs a probability distribution over 51 atoms in $[-10, 10]$; loss is KL divergence.
- **Prioritized Experience Replay (PER)**: priority $\propto |\delta|^\alpha$ ($\alpha = 0.5$), IS correction $\beta = 0.4$.
- **N-step returns**: 3-step bootstrapped targets.
- **Noisy Networks**: NoisyLinear layers ($\sigma_0 = 0.5$) replace ε-greedy.
- **Dueling architecture**: separate value and advantage streams, $Q = V + A - \bar{A}$.

3.4 Limitations

Despite 1 000 training episodes, neither DDQN nor Rainbow achieved consistent long-game performance:

1. **Credit assignment**: a good placement may not produce a line clear for several steps; discounted credit is weak.
2. **Q-function instability**: 64 outputs with a highly variable valid mask at each step.
3. **Sample efficiency**: 2.67M parameters trained on only 1 000 episodes is severely underfit.
4. **Perception overhead**: the network must simultaneously learn piece recognition, valid placements, *and* long-term board value.

4

DEEP VALUE NETWORK (DVN)

4.1 Afterstate Formulation

Instead of learning $Q(s, a)$, the DVN learns a *state value function* $V_\theta(s')$ where s' is the *afterstate*: the board after placement and line clearing. Since the environment precomputes `placements_result`, the agent enumerates all valid afterstates and selects:

$$a^* = \arg\max_a \left[r_a + \gamma \cdot V_\theta(s'_a)\right]$$

Advantages over Q-learning: the network only evaluates boards (not board–action pairs), valid constraints are handled implicitly, and the value function captures board quality independently of the current piece.

4.2 Architecture: Multi-Kernel CNN

The value network `BlockBlastValueNet1PmultikernelFlattenned` uses a bank of eight parallel convolutional branches:

Branch	Kernel	Filters	Wall padding	Output spatial
Square 1×1	$(1, 1)$	1	0	8×8
Square 2×2	$(2, 2)$	8	1	9×9
Square 3×3	$(3, 3)$	16	1	8×8
Square 4×4	$(4, 4)$	16	2	9×9
Square 5×5	$(5, 5)$	16	2	8×8
Square 8×8	$(8, 8)$	64	0	1×1
Horizontal 1×8	$(1, 8)$	4	0	8×1
Vertical 8×1	$(8, 1)$	4	0	1×8

Padding uses `ConstantPad2d` with value **1.0** so the kernel sees filled wall cells outside the board boundary. Line kernels (1×8, 8×1) use no padding.

Each branch: Flatten $\to$ Linear($\cdot \to 16$) $\to$ LeakyReLU $\to$ Linear($16 \to 8$). All eight outputs are concatenated into a **64-dimensional** feature vector.

The role of each branch family:

- 1×1: per-cell occupancy.
- 2×2–5×5: local block patterns matching common piece shapes.
- 8×8: global board summary.
- 1×8, 8×1: full-row and full-column detectors—crucial for identifying nearly-complete lines.

The output of the convolutional branches is fed into a 3-layer MLP head:

$$\text{Linear}(64 \to 64) \to \text{LeakyReLU} \to \text{Linear}(64 \to 16) \to \text{LeakyReLU} \to \text{Linear}(16 \to 1)$$

Total parameters: **42 461** (63× fewer than DDQN/Rainbow).

4.3 Training

4.3.1 • Principle

The DVN is trained via temporal difference learning on afterstates. Let s'_t be the chosen afterstate at time t.

TD Target. For non-terminal transitions:

$$y_t = \max_{a'} \left[r_{t+1,a'} + \gamma \cdot V_{\theta^-}(s'_{t+1,a'}) \right]$$

where θ^- is a target network updated every 200 episodes. For terminal transitions:

$$y_t = \frac{p_{\text{invalid}}}{\gamma}$$

Loss. Huber loss (SmoothL1) with gradient clipping at norm 10.

Protocol.

- 10 000 episodes, max 100 steps/episode.
- ε-greedy: $1.0 \to 0.01$, decay $\times 0.999$/episode.
- Replay buffer: 100 000 transitions, batch size 128.
- Adam, lr $= 10^{-4}$, StepLR ($\gamma_{\text{sched}} = 0.99$, step 1 000).
- Checkpoints every 500 episodes with full training state; logging via Weights & Biases.

4.3.2 • Dynamics

Figure 1 illustrates the evolution of the accumulated reward, temporal difference loss, and episode length over the training process.

../plots/wandb_reward.png

(a) Reward vs. Episode

../plots/wandb_loss.png

(b) Loss vs. Episode

../plots/wandb_eplen.png

(c) Length vs. Episode

Figure 1: Training metrics recorded on Weights & Biases for the 1P DVN.

Due to the ε-greedy schedule, the agent relies heavily on random exploration during the early phase of training. During the first $\sim$3 000 episodes, performance remains relatively low and highly stochastic. The loss initially rises as the replay buffer gathers high-reward transitions (from lines being cleared) and starts bootstrapping higher value targets. As the exploration rate decays and the DVN refines its afterstate evaluations, both the episode length and total reward steadily climb. The metrics demonstrate clear learning progress, reaching convergence at around 10 000 episodes.

4.4 Results 1P

Figure 2 shows the episode return and length distributions for the trained DVN, a greedy baseline, and a random baseline.

../plots/benchmark_dvn_vs_greedy_vs_random_20260313_230003.png

Figure 2: Distribution of episode returns (left) and episode lengths (right) for DVN, greedy, and random agents on the 1P Block Blast environment.

The trained DVN manages to play very good games (reaching around 300 steps in the best cases), giving it a high mean score. However, it exhibits a high variance with a median that is significantly lower than the mean. This is primarily due to the highly stochastic nature of the environment, which inevitably leads to a large number of poorly performing "bad" games.

Agent	Avg. Return	Avg. Steps	Parameters
Random	—	∼5	—
Greedy	—	∼30	—
DVN (untrained)	−815.98	∼20	42 461
DVN (trained, 60 avg)	—	∼60	42 461
DDQN 1P	—	∼20	2 674 464
Rainbow 1P	—	∼20	2 674 464

Table 1: Agent performance summary on the 1P environment.

The trained DVN achieves ∼60 average steps per episode vs. ∼20 for an untrained model. The greedy baseline (always pick highest immediate reward) is shortsighted and over-fills the board; the DVN outperforms it by learning to preserve board space.

4.5 Results 3P

For the 3P variant, a `RoundPlanner3P` wraps the DVN. At each new round it calls `get_t_plus_3_candidates(`γ`)`, scores each candidate by computing the immediate discounted rewards plus the discounted value:

$$\text{score} = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 \cdot V_\theta(s_{t+3})$$

and executes the best 3-action sequence, replanning if an action becomes invalid.

Up to this point, our study has focused on the 1P variant, which significantly simplifies the action space. The core dynamics of *Block Blast*, however, rely on the 3P variant, where three pieces are generated simultaneously and must be placed on the grid to complete a wave. This variant introduces a major combinatorial challenge: the agent is free to choose the order in which the three pieces are played.

4.5.1 • Motivation and Combinatorial Explosion

Transitioning from 1P to 3P is not merely an increase in the number of pieces, it entails an exponential explosion of the search tree. For a given state with three pieces, there are $3! = 6$ possible placement sequences. Assuming each piece has an average of ten valid positions, evaluating all permutations and positions quickly leads to thousands of potential terminal states for a single wave.

The objective of this section is to verify whether our **Deep Value Network (DVN)**, initially trained on the 1P variant to evaluate the structural quality of a grid, can be robustly used as a heuristic in a limited-depth search tree (*lookahead*) for the 3P variant.

4.5.2 • Benchmark Methodology

To quantify the model's performance, we implemented several planning policies and compared them on a rigorous benchmark consisting of 250 episodes with a discount factor $\gamma = 0.99$.

As for the 1P setting, the following agents were evaluated:

- **Random:** Uniformly selects an action among the legal moves. This approach serves as our lower-bound baseline.

- **Greedy (Depth-2 & Depth-3):** A myopic search algorithm that simulates the possibility tree and selects the sequence maximizing the immediate reward (i.e., the score obtained by clearing lines) during the current wave.
- **DVN (Depth-2 & Depth-3):** The agent uses our value network to evaluate potential *afterstates*. The optimal sequence of actions $\mathcal{S}^*$ is chosen by maximizing the sum of immediate rewards and the discounted future value of the resulting state:

$$\mathcal{S}^* = \arg\max_{\mathcal{S}} \left(\sum_{r_t \in \mathcal{S}} \gamma^t r_t + \gamma^k V_\theta(s_{\text{final}}) \right) \tag{2}$$

where V_θ is the value predicted by the network and k is the depth of the simulation within the tree.

4.5.3 • Results and Superiority of the DVN

The performances of the agents at depth 2 (anticipating the placement of 2 pieces) reveal a striking performance gap. The statistics computed over our main sample of 250 episodes are summarized in the table below:

Policy	Mean Reward (Score)	Mean Length (Steps)
DVN depth-2	**23 412.72**	**209.29**
DVN depth-3	**47 469.39**	**223.72**
Greedy depth-2	1 884.90	44.45
Random	25.33	11.93

Table 2: Performance comparison of the agents on the 3P variant (250 episodes, $\gamma = 0.99$).

Several decisive conclusions emerge from these results:

1. **The Failure of the Greedy Approach:** The *Greedy* agent demonstrates that pure short-term reward maximization is deeply suboptimal for *Block Blast*. By focusing entirely on immediate line clears to score points, the *Greedy* policy tends to fragment the free space on the grid. This rapidly makes the board unplayable, leading to premature game overs at around 44 steps on average.
2. **The Geometric Understanding of the DVN:** Conversely, the DVN depth-2 agent survives nearly five times longer (∼209 steps) and achieves a massive average score of over 23,400 points. This proves that the network successfully captured the "intrinsic value" of a board. It is capable of sacrificing immediate rewards (e.g., delaying a line clear) if it identifies that doing so preserves a healthier grid topology, thereby mitigating the risk of a game over.
3. **Impact of Higher Depth (Depth-3):** Expanding the evaluation to depth 3 allows the agent to anticipate the entire wave. Although significantly more computationally expensive due to the massive number of states evaluated in batches by the network, this increased depth should enable the agent to avoid geometric dead ends that only become apparent when placing the third piece of the draw. In practice, we observed significantly higher rewards with this increased depth (the rewards doubled) whereas the length of the episodes remained about the same (which may be due to the small number of episodes used for evaluation at depth 3, due to the computation cost of this policy).

../plots/benchmark_3p_d2_unclipped.png

Figure 3: Performance of the DVN for a depth 2 look ahead

../plots/benchmark_3p_d3_unclipped.png

Figure 4: Performance of the DVN for a depth 3 look ahead

4.5.4 • Computational Bottlenecks and Limitations

While the *Deep Value Network* proves extremely effective at navigating this large combinatorial space, its exhaustive application quickly becomes a computational bottleneck. Performing inference on all *leaf boards* at depth 3 requires evaluating very large tensors by batch on the GPU and limiting ourselves to the much more reasonable depth of 2 for the lookahead might be interesting.

../plots/benchmark_3p_d2_vs_d3_unclipped.png

Figure 5: Performance comparison according to the depth of the lookahead

5 PROXIMAL POLICY OPTIMIZATION (PPO)

5.1 Principle

Instead of learning a value function over states, PPO directly learns a stochastic policy $\pi_\theta(a|s)$ that maps the current board state and available pieces to a probability distribution over valid placements. The observation is encoded through three specialized modules: a BoardEncoder that processes the binary 8×8 grid through a convolutional stack into a 128-d feature vector, a PiecesEncoder that encodes the three 5×5 padded pieces as a 3-channel input into a 64-d vector, and a ScalarEncoder that maps the combo counter and piece usage flags

into a 32-d vector.These representations are concatenated into a 224-d joint embedding and passed through a shared MLP trunk producing a 256-d feature vector, from which a policy head and a value head are derived. The policy is updated by maximizing a clipped surrogate objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t[min(r_t(\theta)\hat{A}^t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}^t)]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio and $\hat{A}_t$ is the advantage estimate computed via GAE using the value head $V_\phi(s)$ trained alongside the policy. The policy head produces logits over the 192 possible placements (3 pieces $\times$ 64 positions), with invalid placements masked to $-\infty$ prior to sampling. During training, actions are sampled stochastically from the resulting distribution; during evaluation, the policy acts greedily by taking the argmax of the masked logits. The advantage of PPO over DVN is that it does not require enumerating all valid afterstates at each step, making it more scalable to the large action space of Block Blast.

5.2 Arborescent search upgrade

To enhance PPO that makes its decisions by a stochastic sampling, we use Monte Carlo Tree Search to have an exhaustive planning of depth 3 in the environment with 3 pieces - it is a loosely inspired from AlphaZero but adapted for Block Blast. The key idea is that the environment is deterministic at each round, it is therefore possible to enumerate all possible placements and to choose the best one, guided by the value network from PPO.

At the beginning of each round (3 pieces to place), all authorized sequences of 3 placements are calculated and scored as the weighted sum of the cumulated reward over the last 3 placements and the value of the terminal state estimated by PPO's value network. The agent then plays the first action of the best sequence, and to continue the round, PPO is simply used in a deterministic manner (with the argmax of the logits). With this upgrade, the mean length of the episodes increases from 25.2 to 36.7, which is a substantial upgrade.

We also tried a more deterministic approach, which turned out to be the most successful: at the beginning of each round, we enumerate all possible 3-placement sequences via `env.get_t_plus_3_candidates()` and score each terminal state `S(t+3)` with the formula:

$$\text{score} = symlog(r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3}) + \alpha \cdot symlog(V_\theta(s_{t+3}))$$

Both terms are kept in symlog space for comparable scales, and because some runs showed that this value could be explosive. The parameter α is a value weight hyperparameter that controls the relative influence of the PPO value network on the score. Hence $\alpha = 0$ means that the agent completely ignores the value network and the model is totally deterministic. Once the best-scoring sequence is identified, all three actions are stored in a queue and executed in order without re-searching at steps 2 and 3. Furthermore, for the sake of efficiency, the value network inference is batched: all candidate terminal boards are stacked into a single tensor and processed in chunks of 512 through the PPO `ActorCritic` network, making the scoring step GPU-efficient despite the potentially large number of candidates.

To determine the optimal α value and to see if there exists a balance between the deterministic and stochastic aspects of this approach, we conducted a sweep over several values, evaluating each configuration over 50 episodes and measuring several metrics.

We observed in practice that the score of the PPO value network is an order of magnitude higher than the score given by the exploration of terminal states. On the figure above, we can note the best results are obtained with $\alpha \in [0.1, 0.5]$, confirming the fact that this problem cannot be simply treated with a brute force approach. We can furthermore note that for such values, the mean length of an episode is around 550, which is the highest score we achieved. However, the research of the terminal state is computationally expensive and take on average

../plots/value_weight_sweep.png

Figure 6: Mean return, median return and mean episode length for different values of alpha

approximately 1100 ms per 3-step sequence.

6 DISCUSSION AND FUTURE WORK

Why DVN outperforms DQN. The DVN decouples *what the board looks like after the move* from *which action achieves it*. The value network only needs to learn a board-to-value mapping—far simpler than the full Q-function which also encodes piece geometry and placement logic.

Why PPO outperforms DVN. While DVN learns a value function solely over afterstates - i.e. the resulting board configuration after a single placement, independently of the remaining pieces or the current position within the round - PPO conditions its value estimate on the full observation: the current board, the three available pieces, the piece usage flags `pieces_used`, and the combo counter. This richer conditioning is particularly beneficial in the context of tree search: when scoring a terminal state s_{t+3}, the PPO value network $V_\phi(s_{t+3})$ has access to the complete context of the position, whereas a DVN-based scorer evaluates the resulting board in isolation. As a result, the value network from PPO provides a more informed heuristic for guiding the search.

Limitations.

- The value function sees only the afterstate board, *not* upcoming pieces.
- The 3P planner's full enumeration is $O(3! \times 64^3)$ in the worst case—slow without batched GPU acceleration.
- The environment's piece generation is purely random and thus highly stochastic, unlike the real game which employs logic to avoid giving overly impossible sequences (e.g., three 3×3 blocks consecutively). This means our environment is unnecessarily harsh on the agent, whose actual performance on the real game might be significantly better.

Future directions.

- Include next-piece information as extra input channels.
- Add a hand-crafted feature branch (holes, row/column fill density, height variance).
- Batch the 3P lookahead on GPU.
- Monte Carlo Tree Search guided by the value function.

7 CONCLUSION

We have presented a reinforcement learning study of Block Blast. Standard DQN approaches (DDQN, Rainbow, ∼2.67M parameters) struggled to converge within the training budget. A Deep Value Network operating on

afterstates—with only 42 461 parameters and a multi-kernel CNN architecture—achieved substantially better performance. The main advantage of the DVN approach is that it does not need to learn the basic mechanics and geometry of the game, since these are handled implicitly by the afterstate generation. However, its true limitation remains the high variance across episodes. Ultimately, what truly allowed the agent to progress from an average of ∼30 steps to ∼60 steps was the proper design and tuning of the reward shaping.

A
MODEL ARCHITECTURES

A.1 BlockBlastCNNNet1P (DDQN / Rainbow)

```
Board + Valid Mask (2, 8, 8)
  -> Conv2d(2->32, k=3, pad=1) -> ReLU
  -> Conv2d(32->64, k=3, pad=1) -> ReLU
  -> Flatten -> 4096

Piece (1, 5, 5)
  -> Conv2d(1->32, k=3, pad=1) -> ReLU
  -> Flatten -> 800

Concat [4096 + 800] = 4896
  -> Linear(4896->512) -> ReLU
  -> Linear(512->256) -> ReLU
  -> Linear(256->64)   [Q-values for 64 actions]
```

A.2 BlockBlastValueNet1PmultikernelFlattened (DVN)

```
Board (1, 8, 8) -- 8 parallel branches:

  Branch k: ConstantPad2d(pad_k, val=1.0)
         -> Conv2d(1->filters_k, kernel_k)
         -> LeakyReLU -> Flatten
         -> Linear(flat_k -> 16) -> LeakyReLU
         -> Linear(16 -> 8)

Concat [8 branches x 8] = 64
  -> Linear(64->64) -> LeakyReLU
  -> Linear(64->16) -> LeakyReLU
  -> Linear(16->1)   [V(s')]
```